

Nome: Luiza de Jesus Lopes

NºUSP: 12822076

EP 2 - Introdução à Análise de Algoritmos - ACH2002

Relatório de comparação empírica de algoritmos de ordenação

Este relatório tem como objetivo comparar e analisar o desempenho dos sete algoritmos de ordenação implementados em linguagem C, são eles: InsertionSort, SelectionSort, BubbleSort, ShellSort, MergeSort, HeapSort e QuickSort. As análises serão feitas a partir das contagens de tempo de execução, número de comparações e número de movimentações em cada algoritmo com conjunto de dados de tamanhos diferentes. Para que as análises sejam o mais precisas possível, todas as contagens serão realizadas em apenas uma máquina e sem nenhum outro processo em execução. Seus atributos são:

Nome da máquina: Dell Vostro 15 3510

Processador: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz 2.42 GHz (processador baseado em x64)

RAM: 8,00 GB

Sistema Operacional: Windows 11

Serão apresentados agora os gráficos e tabelas acerca do tempo de execução, número de movimentações e número de comparações, variando pelo número de chaves (100, 1000 e 10000), tamanho do *campoDaEstrutura* (1 e 1000) das estruturas de registro e ordem das chaves (crescente, decrescente e aleatória). É importante destacar que o tempo de execução estará sendo medido em microssegundos (ms).

Análise do tempo de execução dos algoritmos com tamanho do *campoDaEstrutura* = 1 e número de chaves = 100, 1.000 e 1.000:

- **Ordem Crescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	2 ms	6 ms	54 ms
SelectionSort	16 ms	1.013 ms	104.977 ms
BubbleSort	1 ms	5 ms	32 ms
ShellSort	4 ms	35 ms	439 ms
MergeSort	9 ms	84 ms	870 ms
HeapSort	10 ms	158 ms	1.590 ms
QuickSort	45 ms	2.482 ms	233.992 ms

Tabela 1 - Tempo de execução por chaves - Ordem crescente

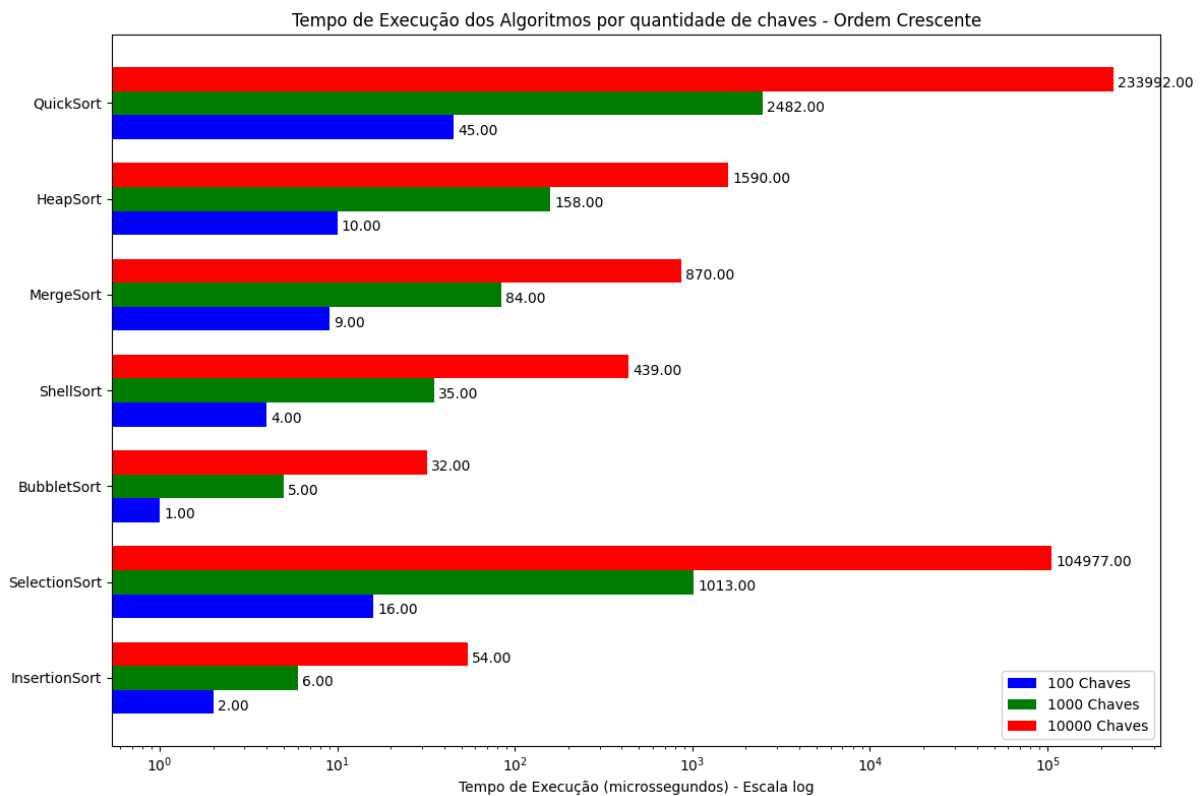


Gráfico 1 - Tempo de execução por chaves - Ordem crescente

- **Ordem Decrescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	17 ms	1.420 ms	141.643 ms
SelectionSort	21 ms	1.526 ms	133.244 ms
BubbleSort	27 ms	2.226 ms	226.764 ms
ShellSort	5 ms	71 ms	660 ms
MergeSort	6 ms	70 ms	831 ms
HeapSort	10 ms	124 ms	1.600 ms
QuickSort	28 ms	1.683 ms	173.924 ms

Tabela 2 - Tempo de execução por chaves - Ordem decrescente

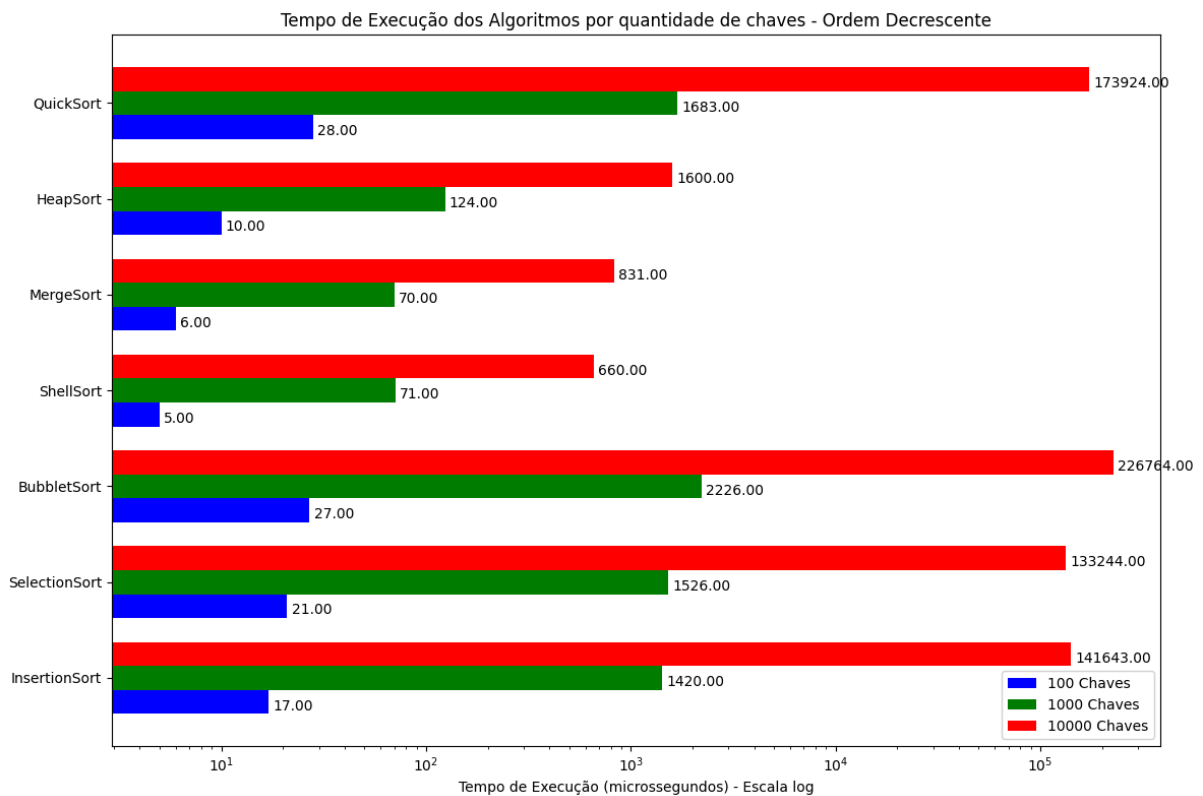


Gráfico 2 - Tempo de execução por chaves - Ordem decrescente

- **Ordem Aleatória**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	9 ms	946 ms	70.367 ms
SelectionSort	28 ms	1.080 ms	1.081 ms
BubbleSort	23 ms	2.053 ms	226.260 ms
ShellSort	9 ms	129 ms	2.019 ms
MergeSort	10 ms	113 ms	1.606 ms
HeapSort	15 ms	194 ms	2.158 ms
QuickSort	9 ms	98 ms	1.258 ms

Tabela 3 - Tempo de execução por chaves - Ordem aleatória

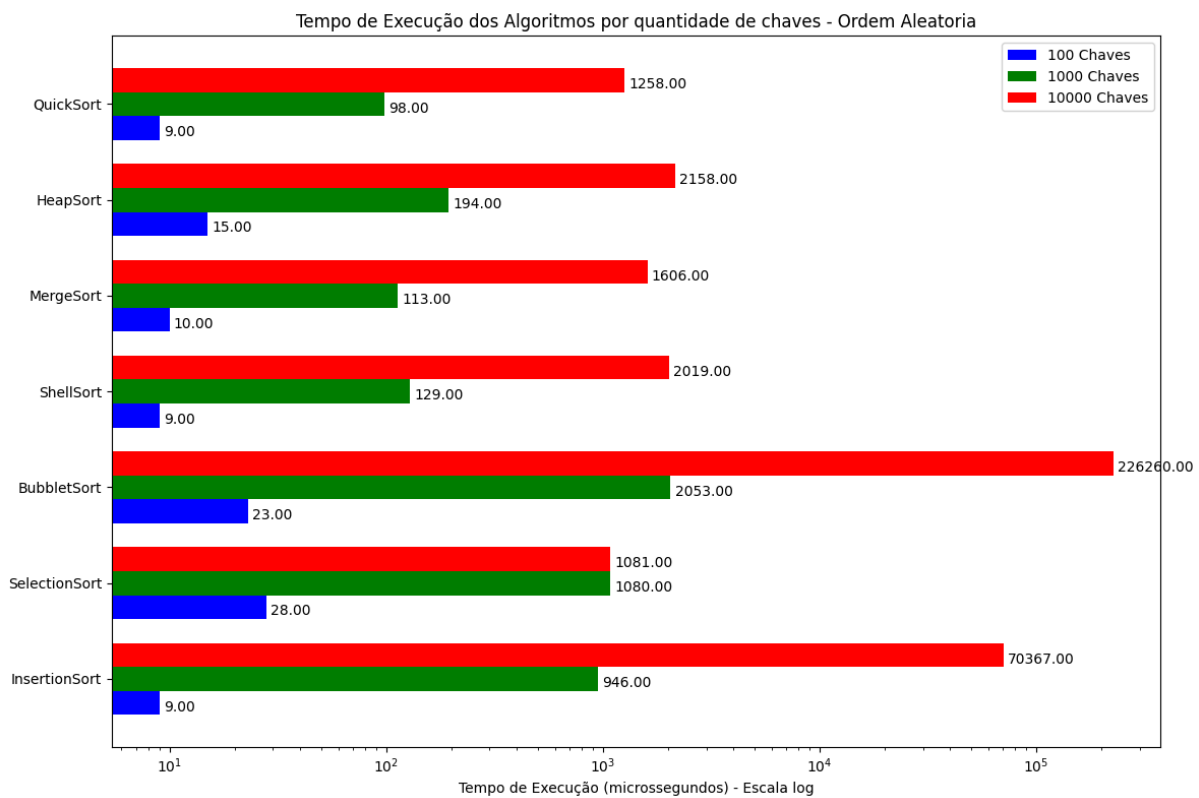


Gráfico 3 - Tempo de execução por chaves - Ordem aleatória

Análise do tempo de execução dos algoritmos com tamanho do *campoDaEstrutura* = 1 e 1.000 e número de chaves = 100:

- **Ordem Crescente**

Algoritmo	<i>campoDaEstrutura</i> = 1	<i>campoDaEstrutura</i> = 1.000
InsertionSort	1 ms	6 ms
SelectionSort	13 ms	14 ms
BubbleSort	1 ms	4 ms
ShellSort	3 ms	5 ms
MergeSort	7 ms	11 ms
HeapSort	10 ms	16 ms
QuickSort	29 ms	32 ms

Tabela 4 - Tempo de execução por quantidade de registros na estrutura - Ordem crescente

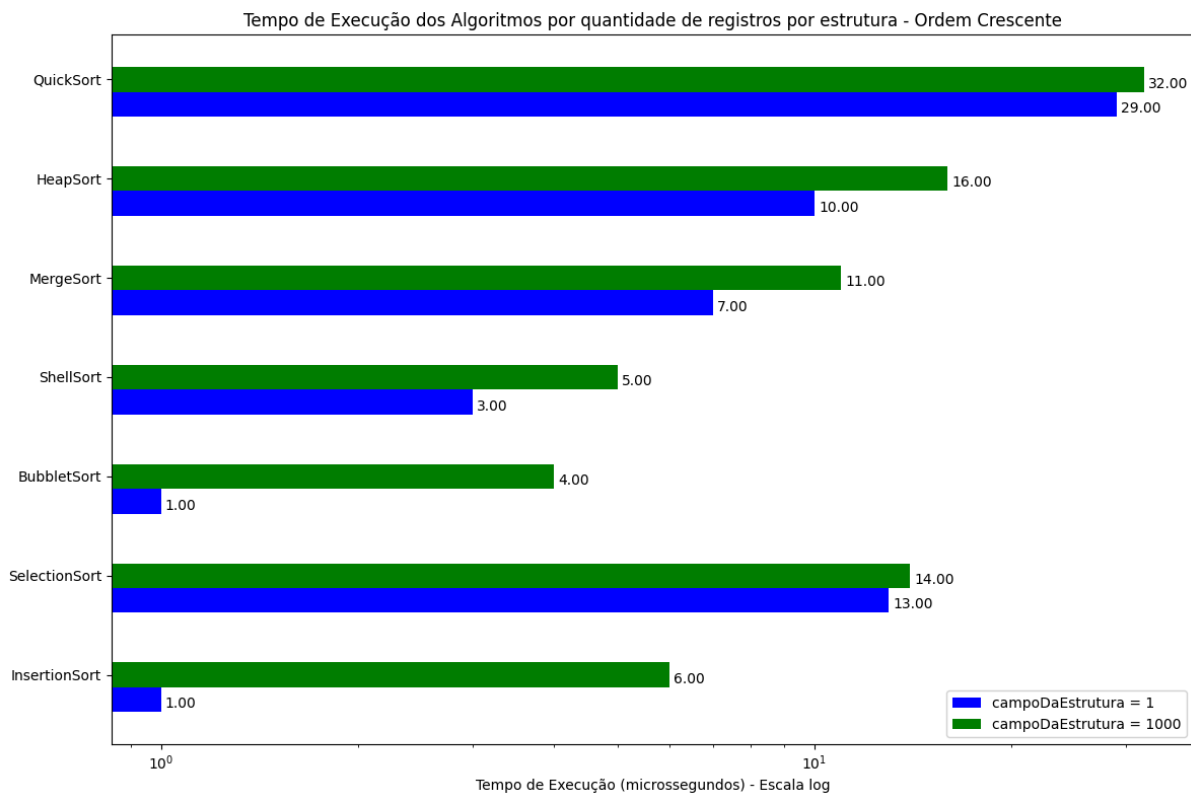


Gráfico 4 - Tempo de execução por quantidade de registros na estrutura - Ordem crescente

- **Ordem Decrescente**

Algoritmo	<i>campoDaEstrutura = 1</i>	<i>campoDaEstrutura = 1.000</i>
InsertionSort	16 ms	22 ms
SelectionSort	15 ms	18 ms
BubbleSort	23 ms	31 ms
ShellSort	5 ms	7 ms
MergeSort	5 ms	12 ms
HeapSort	9 ms	11 ms
QuickSort	20 ms	27 ms

Tabela 5 - Registros por tempo de execução - Ordem decrescente

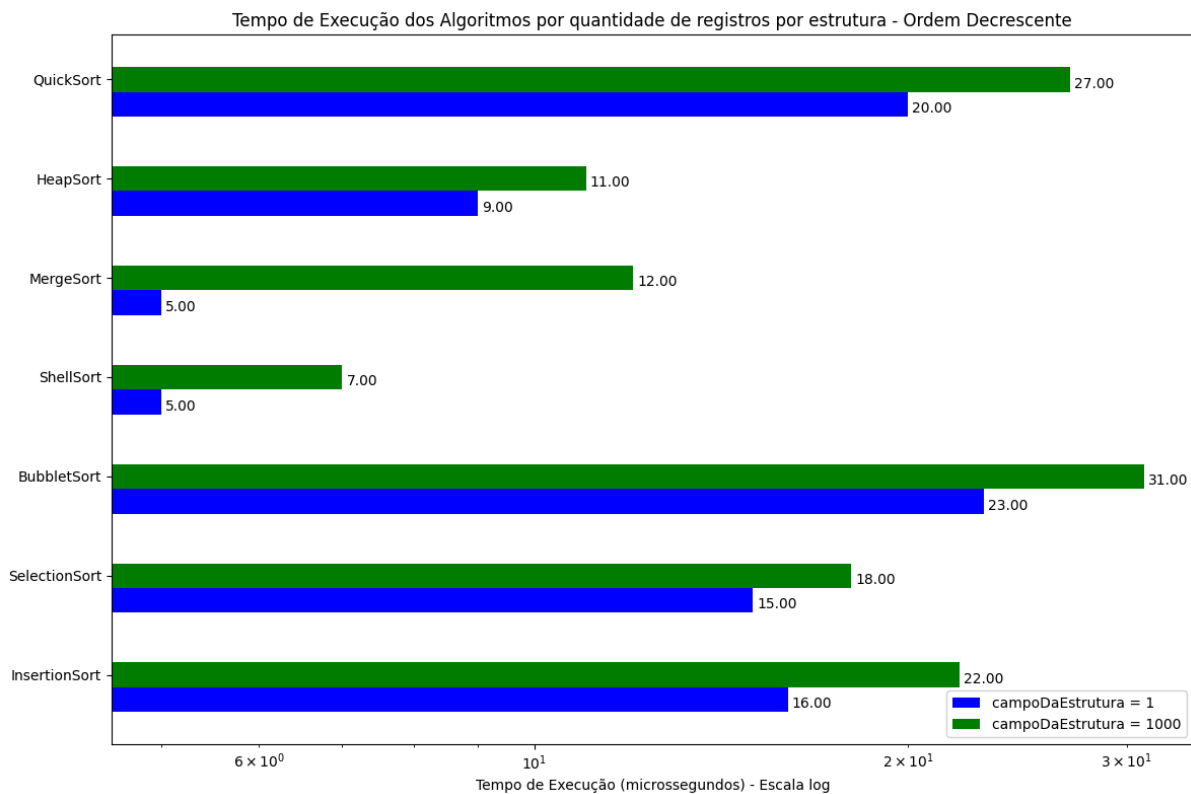


Gráfico 5 - Tempo de execução por quantidade de registros na estrutura - Ordem decrescente

- **Ordem Aleatória**

Algoritmo	<i>campoDaEstrutura = 1</i>	<i>campoDaEstrutura = 1.000</i>
InsertionSort	12 ms	16 ms
SelectionSort	20 ms	31 ms
BubbleSort	44 ms	50 ms
ShellSort	10 ms	18 ms
MergeSort	15 ms	29 ms
HeapSort	11 ms	36 ms
QuickSort	12 ms	18 ms

Tabela 6 - Registros por tempo de execução - Ordem aleatória

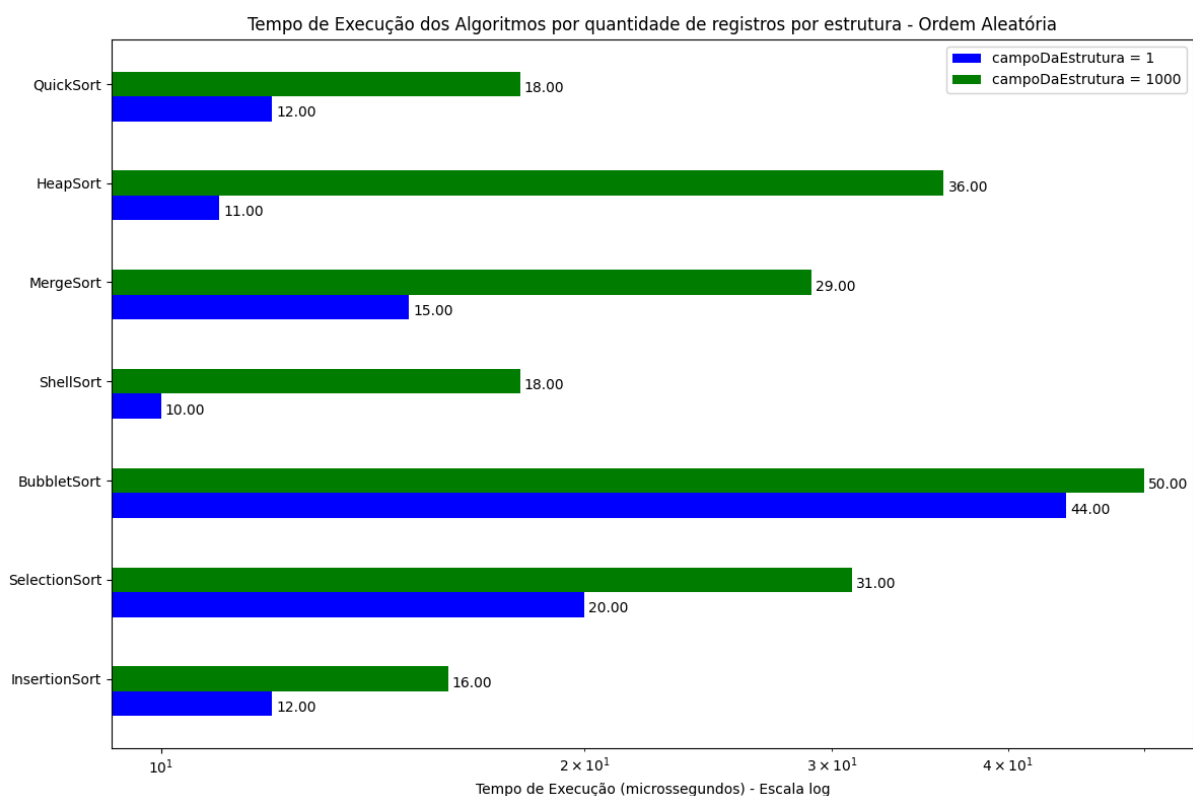


Gráfico 6 - Tempo de execução por quantidade de registros na estrutura - Ordem aleatória

Análise do número de comparações dos algoritmos com número de chaves = 100, 1.000 e 10.000 e tamanho do *campoDaEstrutura* = 1.

- Ordem Crescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	198	1.998	19.998
SelectionSort	5.049	500.499	50.004.999
BubbleSort	100	1.000	10.000
ShellSort	890	12.922	170.497
MergeSort	1.913	26.079	367.139
HeapSort	795	12.963	180.584
QuickSort	4.950	499.500	49.995.000

Tabela 7 - Número de comparações por número de chaves - Ordem crescente

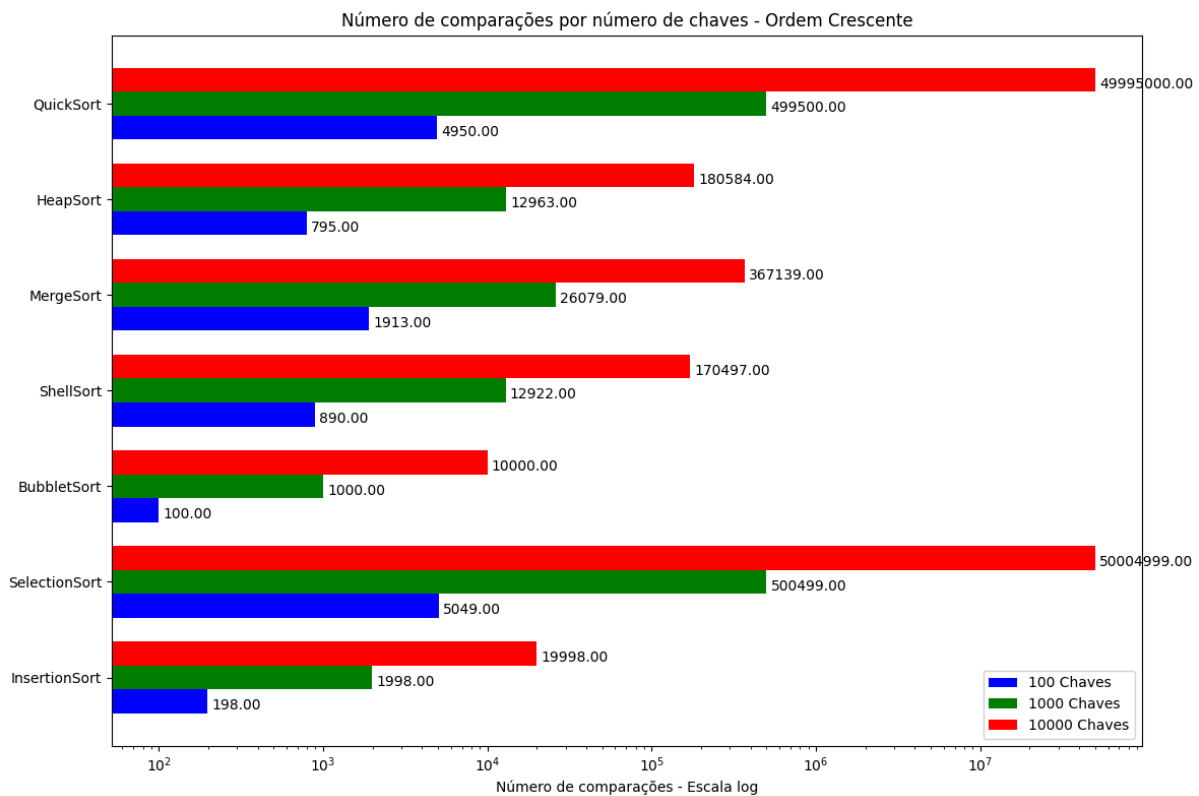


Gráfico 7 - Número de comparações por número de chaves - Ordem crescente

- **Ordem Decrescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	5.049	500.499	50.004.999
SelectionSort	7.599	750.999	75.009.999
BubbleSort	9.999	999.999	99.999.999
ShellSort	1.048	16.015	215.444
MergeSort	1.541	21.027	295.427
HeapSort	961	10.340	153.619
QuickSort	4.950	499.500	49.995.000

Tabela 8 - Número de comparações por número de chaves - Ordem decrescente

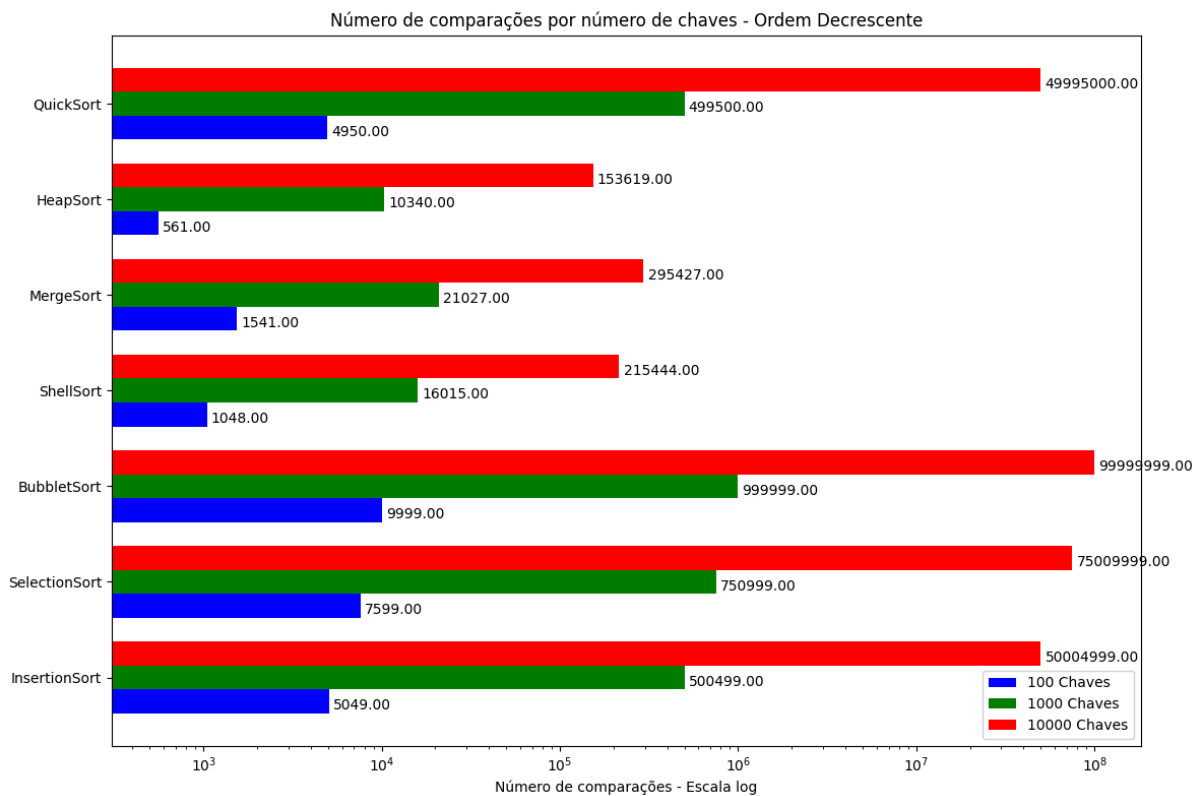


Gráfico 8 - Número de comparações por número de chaves - Ordem decrescente

- **Ordem Aleatória**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	2.368	250.293	25.024.261
SelectionSort	5.491	506.926	50.093.738
BubbleSort	7.105	746.784	74.998.394
ShellSort	1.283	21.524	323.583
MergeSort	1.853	25.413	360.827
HeapSort	700	11.688	166.216
QuickSort	607	111.112	153.913

Tabela 9 - Número de comparações por número de chaves - Ordem aleatória

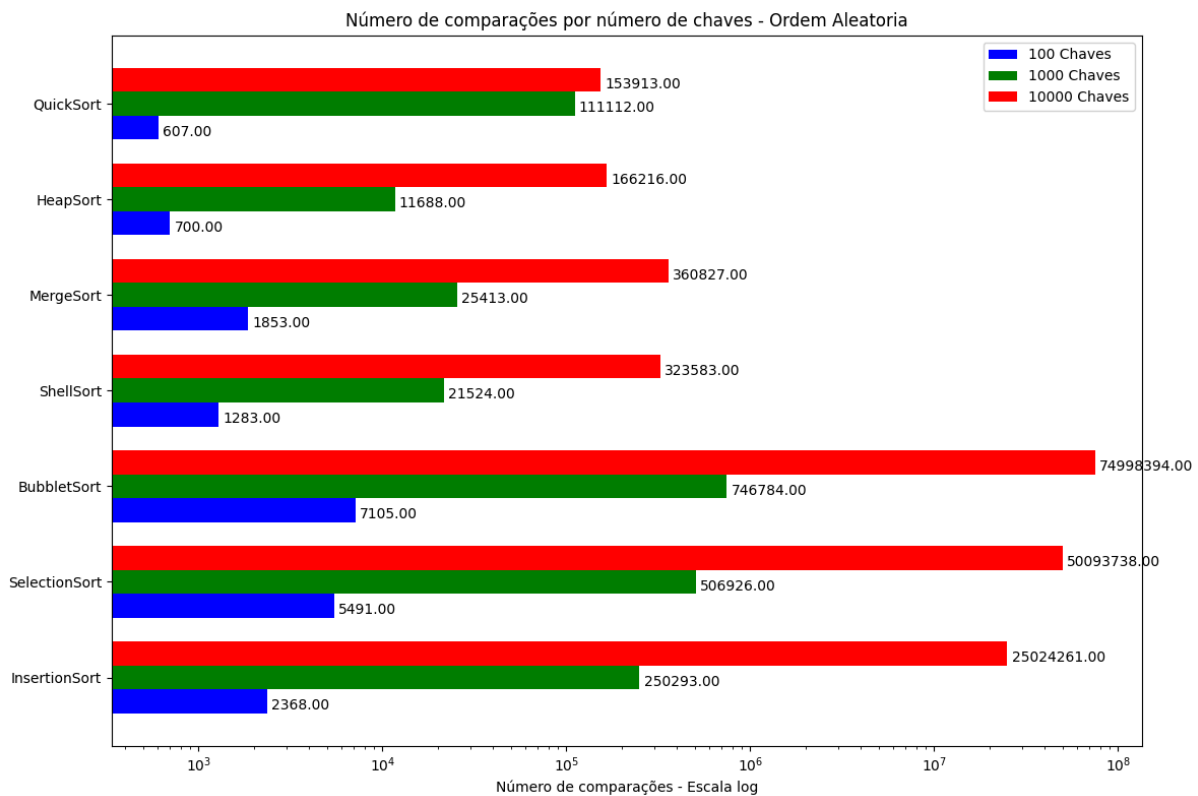


Gráfico 9 - Número de comparações por número de chaves - Ordem aleatória

Análise do número de movimentações dos algoritmos com número de chaves = 100, 1.000 e 10.000 e tamanho do *campoDaEstrutura* = 1.

- **Ordem Crescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	198	1.998	19.998
SelectionSort	99	999	9.999
BubbleSort	0	0	0
ShellSort	884	12.914	170.486
MergeSort	1.432	20.016	285.408
HeapSort	1.920	29.124	395.858
QuickSort	15.147	1.501.497	150.014.997

Tabela 10 - Número de movimentações por número de chaves - Ordem crescente

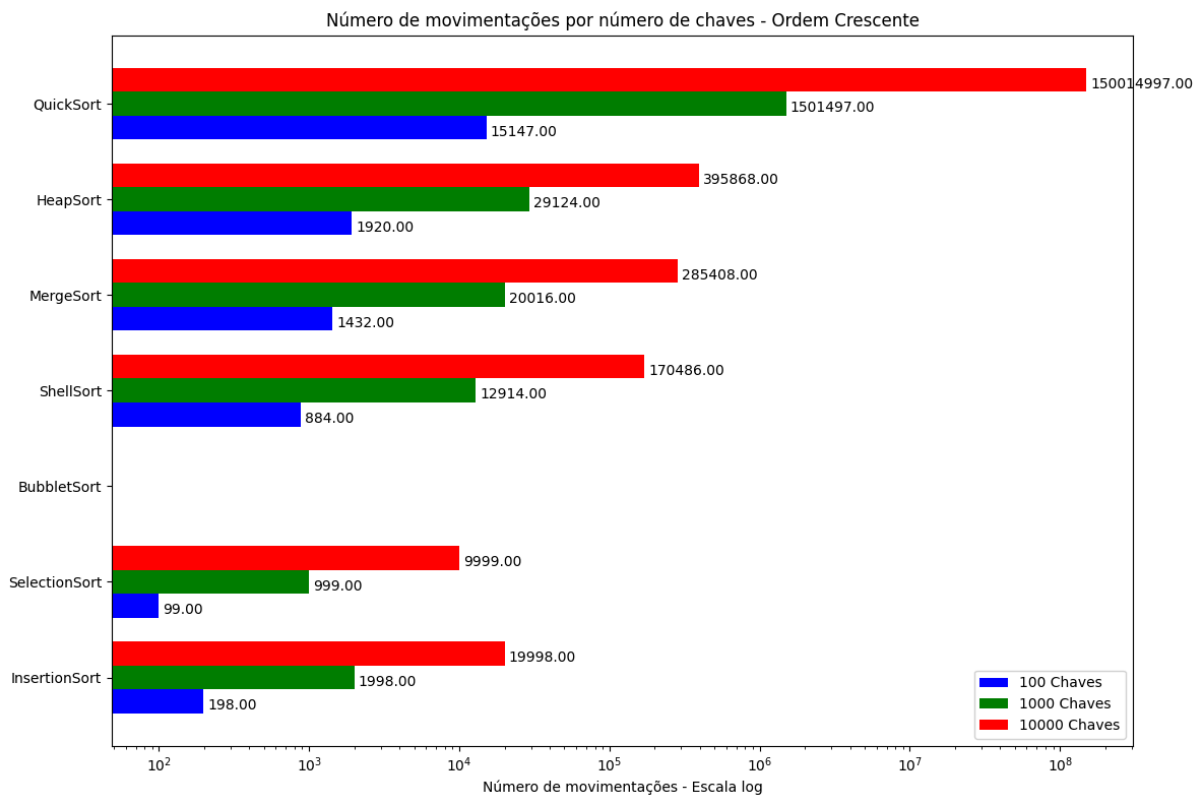


Gráfico 10 - Número de movimentações por número de chaves - Ordem crescente

- **Ordem Decrescente**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	5.148	501.498	50.014.998
SelectionSort	2.749	252.499	25.024.999
BubbleSort	14.850	1.498.500	149.985.000
ShellSort	1.344	20.754	277.894
MergeSort	1.432	20.016	285.408
HeapSort	1.548	24.948	350.088
QuickSort	7.647	751.497	75.014.997

Tabela 11 - Número de movimentações por número de chaves - Ordem decrescente

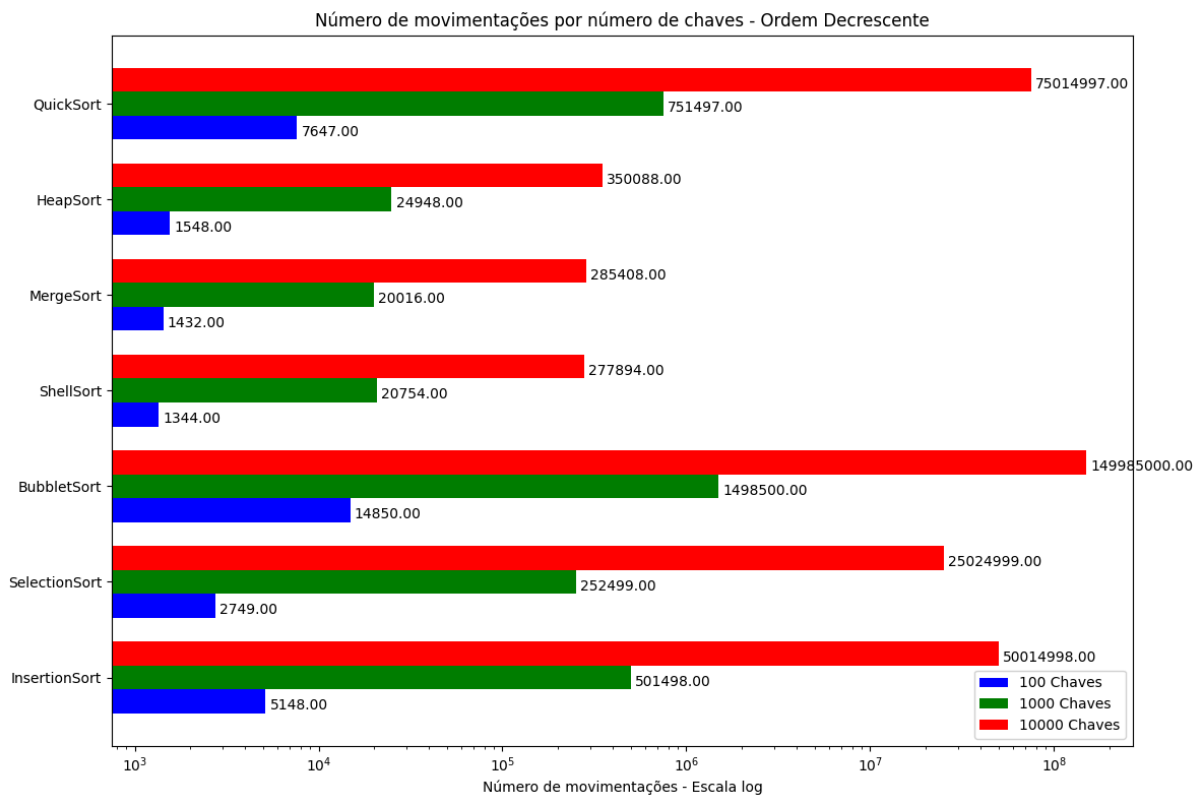


Gráfico 11 - Número de movimentações por número de chaves - Ordem decrescente

- **Ordem Aleatória**

Algoritmo	100 chaves	1.000 chaves	10.000 chaves
InsertionSort	2.373	250.298	25.024.270
SelectionSort	735	9.406	118.714
BubbleSort	6.525	744.900	75.012.816
ShellSort	1.730	30.990	485.230
MergeSort	1.432	20.016	285.408
HeapSort	1.785	27.216	372.552
QuickSort	1.284	20.238	245.772

Tabela 12 - Número de movimentações por número de chaves - Ordem aleatória

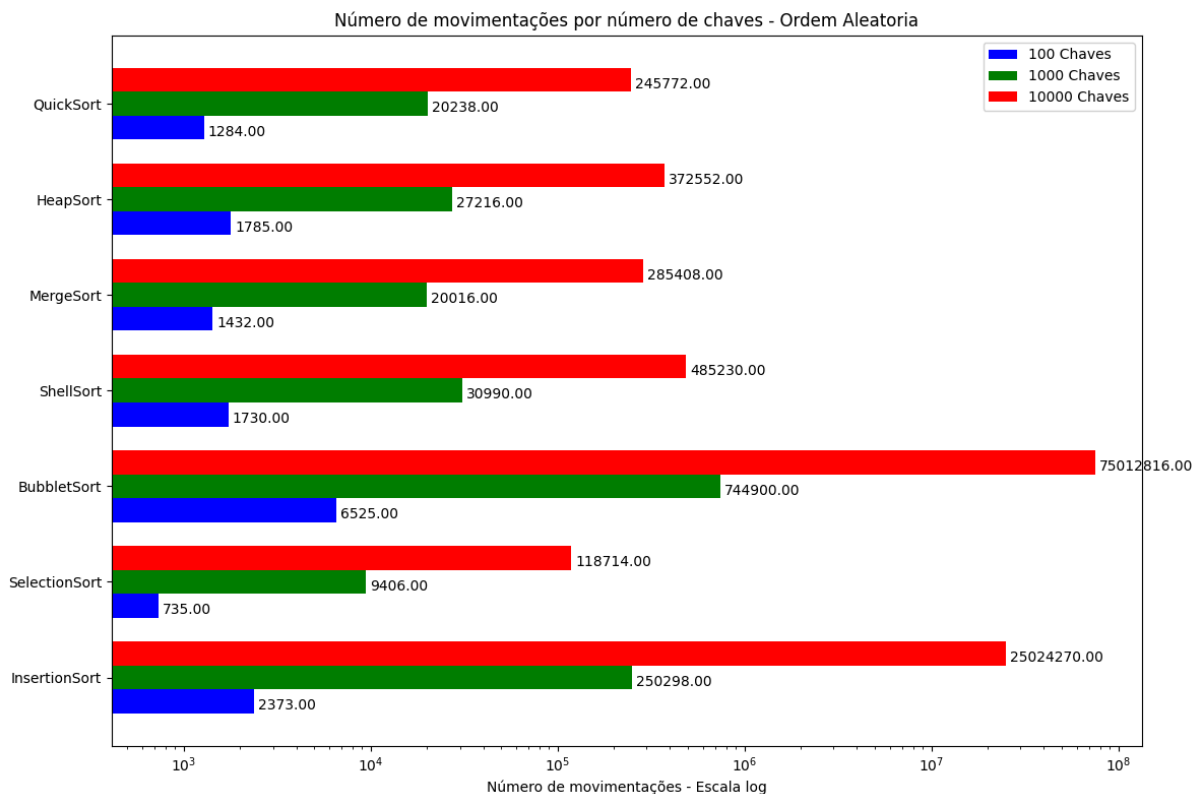


Gráfico 12 - Número de movimentações por número de chaves - Ordem aleatória

Análises e discussões sobre os algoritmos e gráficos gerados

Os sete algoritmos de ordenação trabalhados na coleta de dados foram implementados de forma a ser possível medir seu desempenho em tempo de execução, comparações entre variáveis e movimentações na memória, dessa forma, conseguimos analisar de forma mais abrangente a eficiência de cada algoritmo em diferentes contextos, seja relacionando a quantidade de chaves em cada estrutura ou o tamanho dos registros. Nesse sentido, a tarefa se tornou ainda mais pragmática ao aplicá-la em situações reais onde não trabalhamos somente com uma lista simples de chaves, mas também com diversos campos de dados que acompanham as chaves, portanto, antes de executar cada algoritmo e coletar os dados de interesse, algumas perguntas surgiram, como “entre os algoritmos analisados, existe um mais eficiente para a maioria dos contextos?” e “existe um melhor entre todos os sete?”. Até o momento, sabe-se que os algoritmos InsertionSort, SelectionSort e BubbleSort são os mais simples e não tão eficientes para grandes registros, ao contrário do ShellSort, MergeSort, HeapSort e QuickSort.

Num primeiro momento, foram analisados os resultados de tempo de execução dos algoritmos utilizando a estrutura de tamanho de registro igual a 1, diversificando somente a quantidade de chaves que, como esperado, interferem no tempo de forma proporcional. Para uma lista de ordem crescente, o algoritmo de BubbleSort demonstrou maior rapidez nas 3 quantidades de chaves, enquanto o QuickSort obteve pior desempenho neste mesmo contexto. Até então são resultados esperados visto que QuickSort possui uma funcionalidade mais robusta com ordenações recursivas, precisando de mais tempo de execução, enquanto BubbleSort e InsertionSort, que ficou em segundo lugar no

desempenho, são lógicas elementares que demandam menos processamento. Já no contexto de uma lista em ordem decrescente o algoritmo que melhor se destacou foi o ShellSort, explicando-se pelo fato de ser o algoritmo mais eficiente em complexidade quadrática ($O(n^2)$), sendo este contexto o pior caso possível. Mas outro algoritmo que também obteve um ótimo desempenho foi o MergeSort, executando 10.000 chaves em 831 ms, justamente por ter uma abordagem muito boa para listas grandes, ao contrário do BubbleSort que, por sua vez, teve o pior desempenho executando o mesmo número de chaves em 226.764 ms. Partindo para um caso menos caótico, os processamentos para a lista de ordem aleatória mostraram resultados medianos e variados, tendo o BubbleSort e InsertionSort como os piores se tratando de 10.000 chaves na lista, enquanto que, no geral, todos se saíram bem com 100 chaves, entretanto, o SelectionSort chamou a atenção ao mostrar tempos de execução praticamente iguais para 1.000 e 10.000 chaves: 1.080 e 1.081, respectivamente, mostrando um desempenho ruim na primeira quantidade, mas ótimo na segunda. Assim, é possível supor que os resultados semelhantes ocorreram por coincidência, mas hipóteses como a disposição dos elementos estarem, respectivamente, piores e melhores para a ordenação podem ser mais realistas.

Seguindo para o segundo conjunto de dados, foi analisado o tempo de execução dos algoritmos em relação ao tamanho dos registros considerando listas com 100 chaves. Em listas de ordem crescente houve uma diversificação de proporcionalidade de resultados em cada algoritmo, por exemplo, o InsertionSort e BubbleSort tiveram discrepância entre seus resultados individuais, assim era esperado para os outros cinco, o que se deu foram resultados próximos em cada algoritmo, principalmente para o QuickSort e SelectionSort que, respectivamente, se explicam por sempre realizar $O(n^2)$ comparações, mesmo nesse contexto de lista já ordenada, e por realizar diversas varreduras no vetor. Já para as listas de ordem decrescente, observou-se um ótimo desempenho do ShellSort e do HeapSort, pois nos dois tamanhos obtiveram os menores resultados de tempo, enquanto QuickSort e BubbleSort foram piores comparando, mas ainda assim não foram péssimos. Para as listas de ordem aleatória, todos os algoritmos tiveram resultados de tempo já esperados com grande discrepância entre os tamanhos de registro, com exceção do BubbleSort que teve maior tempo em ambos os tamanhos.

Partindo para a terceira coleta de dados, foram comparados os números de comparações de cada algoritmo nas listas com 100, 1.000 e 10.000 chaves de tamanho de registro igual a 1 que, embora seja pequeno, foi observado grandes resultados. Para as listas de ordem crescente, os algoritmos que tiveram maiores números de comparação nos três contextos foram o SelectionSort e QuickSort. Faz sentido que o SelectionSort tenha tido essa performance dado a sua lógica de funcionamento que se baseia em escolher o menor valor da sub-lista não-ordenada à esquerda e trocar com o primeiro elemento da sub-lista, para isso, há algumas comparações ocorrendo, já o QuickSort também possui diversas comparações em sua lógica de ordenação recursiva. Nas listas de ordem decrescente os números aumentaram considerando o contexto de pior caso possível. Os algoritmos elementares InsertionSort, SelectionSort e BubbleSort foram os que tiveram maiores números de comparações nas três quantidades de chaves, provavelmente porque são os menos eficientes, seguindo do QuickSort. Entre as listas de ordem aleatória, nota-se a permanência desse grande volume de comparações, exceto com QuickSort que apresentou números mais baixos e equiparados com os outros algoritmos de maior eficiência para grandes registros e listas desordenadas.

Agora falando sobre o último conjunto de dados coletados, fez-se a análise sobre os números de movimentações feitas na memória considerando listas com 100, 1.000 e 10.000 chaves de tamanho de registro igual a 1. Nas listas de ordem crescente, o BubbleSort foi o melhor em economia de memória ao não fazer movimentação alguma, já o QuickSort teve valores expressivamente grandes por ter realizar diversas movimentações em seu funcionamento, enquanto os demais algoritmos obtiveram resultados medianos que não se diferiram muito. Para o pior caso (ordem decrescente), os três números para cada algoritmo foram proporcionais entre si e o HeapSort, MergeSort e ShellSort tiveram os menores resultados nos três contextos, o que é um tanto surpreendente pensando no HeapSort pois este devido a sua repetição de extração do valor máximo para o final do vetor, já o BubbleSort, SelectionSort e InsertionSort tiveram os maiores valores novamente. Agora olhando para as listas em ordem aleatória, o comportamento do QuickSort foi melhor em detrimento do BubbleSort e InsertionSort que continuam com valores grandes, sendo o SelectionSort uma exceção.

Dessa forma, percebe-se que os algoritmos mais simples como InsertionSort e BubbleSort demonstraram maior desempenho em tempo de execução em pequenas listas e aleatórias, enquanto o SelectionSort mostrou boa performance somente nos valores em listas ordenadas ou aleatórias. Já os mais complexos como MergeSort, HeapSort e QuickSort se destacaram em cenários com listas maiores e desordenadas. Esta variação de desempenho se estendeu às operações internas, com alguns algoritmos realizando mais comparações e outros mais movimentos, ressaltando a importância de considerar não apenas o tempo de execução, mas também esses fatores para avaliar se um algoritmo é interessante para a aplicação num determinado contexto ou não. Ademais, a ordem inicial dos dados também se mostrou importante para a análise, com algoritmos apresentando desempenhos interessantes em cenários particulares, como o BubbleSort em listas já ordenadas mantendo valores baixos para todas os conjuntos. Portanto, considerando todas as análises feitas, fica evidente que não existe um melhor algoritmo para todas as situações, como foi questionado inicialmente, pois o desempenho varia significativamente conforme o tamanho da entrada, a ordem inicial das chaves nas listas e o tamanho dos registros de cada estrutura.

Fontes de estudos

ESTRATÉGIA CONCURSOS. **Algoritmo de Ordenação por Inserção** – Concurso BB (TI). 2023. Disponível em:

<https://www.estrategiaconcursos.com.br/blog/algoritmo-ordenacao-insercao/>. Acesso em: 10 nov. 2024.

VISUALGO. **VisuAlgo - visualising data structures and algorithms through animation**. 2024. Disponível em: <https://visualgo.net/en>. Acesso em: 10 nov. 2024.

UNIVERSIDADE FEDERAL DE MINAS GERAIS. Quicksort. 2024. Disponível em: <https://homepages.dcc.ufmg.br/~clodoveu/files/AEDS2/AEDS2.12%20Quicksort.pdf>. Acesso em: 12 nov. 2024.

CYBERINI. **Quicksort (análise e implementações)**. 2018. Disponível em: <https://www.blogcyberini.com/2018/08/quicksort-analise-e-implementacoes.html>. Acesso em: 16 nov. 2024.

TREINAWEB. **Conheça os principais algoritmos de ordenação**. 2024. Disponível em: <https://www.treinaweb.com.br/blog/conheca-os-principais-algoritmos-de-ordenacao>. Acesso em: 10 nov. 2024.

DEVMEDIA. **Algoritmos de Ordenação: Análise e Comparação**. 2024. Disponível em: <https://www.devmedia.com.br/algoritmos-de-ordenacao-analise-e-comparacao/28261>.

Acesso em: 16 nov. 2024.

DESENVOLVENDO SOFTWARE. **Selection Sort**. 2024. Disponível em: <https://desenvolvendosoftware.com.br/algoritmos/ordenacao/selection-sort.html>. Acesso em: 16 nov. 2024.